

Федеральное государственное автономное образовательное учреждение
высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ОТЧЕТ ПО ПРОЕКТУ
«Клиент-серверное приложение для решения математических задач»
По дисциплине «Технологии и методы программирования»

Выполнили студенты группы 251-372

Андреев И.А.

Галушкина В.Р.

Лукашин Т.Е.

Савинова Я.И.

1 Введение

Цель работы: разработать клиент-серверное приложение, реализующее решение численного математического задания – нахождение корней кусочной нелинейной функции – с полноценным пользовательским интерфейсом, регистрацией пользователей, подтверждением по электронной почте, хранением статистики и визуализацией графиков.

Проект включает несколько самостоятельных компонентов:

- EchoServer – консольный TCP-сервер на Qt (вычисления, БД, SMTP)
- Client – графический Qt-клиент с несколькими экранами
- Tests – набор модульных тестов на Qt Test
- Docker-окружение – для изолированного запуска сервера и MailHog

Проект демонстрирует практическое применение технологий объектно-ориентированного программирования, сетевого взаимодействия, работы с базами данных, автоматического тестирования и контейнеризации.

2 Постановка задачи

2.1 Математическая постановка

Требуется численно найти все корни уравнения $f(x) = 0$ для кусочно-определённой функции:

$$f(x) = \begin{cases} a \cdot \ln(x + 1), & \text{при } -1 < x < 0 \\ x^3 - 3x + 1, & \text{при } 0 \leq x \leq 2 \\ 1/(x - 2) - 1, & \text{при } x > 2 \end{cases}$$

При базовых параметрах $a = 1$, $b = 1$ функция имеет три корня:

- $x_1 \approx 0.347$ – сегмент $[0, 2]$
- $x_2 \approx 1.532$ – сегмент $[0, 2]$
- $x_3 = 3.0$ – сегмент $(2, +\infty)$

Корни находятся численно методом бисекции (деления отрезка пополам) с точностью $\varepsilon = 10^{-6}$.

2.2. Функциональные требования

К системе были предъявлены следующие функциональные требования:

1. Регистрация нового пользователя с подтверждением адреса электронной почты.
2. Авторизация зарегистрированного и подтверждённого пользователя.
3. Выбор варианта задачи из предложенного набора.
4. Ввод ответа пользователем и проверка на сервере с допуском ± 0.001 .
5. Хранение статистики: количество попыток, правильных ответов, очков.

6. Отображение статистики в интерфейсе клиента.
7. Визуализация графиков исследуемых кусочных функций с обозначением найденных корней.
8. Изолированное развёртывание сервера в Docker-контейнере.

2.3. Нефункциональные требования

- Протокол обмена – текстовый, сообщения разделяются символом \n.
- Хранилище данных – SQLite (встроенная СУБД, не требует отдельного сервера).
- Безопасность паролей – хэширование SHA-256.
- Архитектура – паттерн MVC как на сервере, так и на клиенте.
- Одиночки (Singleton) – для объектов БД и TCP-клиента.
- Покрывание unit-тестами – не менее 10 тест-кейсов для математических функций и протокола.

3 Архитектура системы

3.1 Общая схема

Система построена по клиент-серверной архитектуре с использованием паттерна MVC (Model-View-Controller) и работает через TCP-протокол на порту 33333.

Компоненты системы:

1. Клиент (Client, Qt GUI)
 - Графическое приложение на Qt
 - Обеспечивает интерфейс для: авторизации, регистрации, решения задач, просмотра статистики и графиков

– Включает окна: LoginWindow, RegisterWindow, TaskWindow, GraphWindow

2. Сервер (EchoServer)

- Консольное приложение на C++
- Принимает подключения на порту 33333
- Обрабатывает команды клиента

3. Бизнес-логика

- Вычисление кусочной функции $f(x)$
- Поиск корней
- Проверка правильности ответов пользователей

4. База данных (DataBase)

- SQLite
- Таблицы: Users (логин, пароль, email, подтверждение), Stats

(логин, задача, результат, очки)

5. Email сервис (EmailSender)

- Отправка кодов подтверждения через SMTP
- Подключение к MailHog (порт 1025)

Взаимодействие:

Клиент (GUI) $\longleftrightarrow$ TCP:33333 $\longleftrightarrow$ Сервер $\longleftrightarrow$ База данных (SQLite)



EmailSender (порт 1025)

3.2 Серверная часть

Сервер организован по паттерну MVC:

Слой	Файлы	За что отвечает
View	mytcpserver.h / .cpp	Приём TCP-соединений, чтение/запись сокетов, буферизация входящих данных

Controller	functionsforserver.h / .cpp	Парсинг протокола, математические вычисления, маршрутизация команд
Model	database.h / .cpp	SQLite через Qt SQL, Singleton, регистрация, авторизация, статистика
Extra	emailsender.h / .cpp	Отправка кода подтверждения по MailHog

Класс MyTcpServer

Класс наследует QObject и использует QTcpServer для прослушивания порта 33333. Каждому подключённому клиенту сопоставляется отдельный буфер накопления (QMap<QTcpSocket*, QString>), что позволяет корректно обрабатывать фрагментированные TCP-сообщения.

Функции сервера (functionsforserver)

Центральная функция parsing() принимает строку сообщения и возвращает строку ответа. Внутри реализована диспетчеризация по первому ключевому слову команды:

Auth – проверка логина и пароля, проверка подтверждения email

reg – регистрация пользователя, генерация кода, отправка письма

confirm – подтверждение email по коду из письма

stat – запрос статистики из БД

check – проверка ответа пользователя на математическую задачу

getparams / setparams – получение и изменение параметров функции

Математическая часть включает: computeF(x) – вычисление значения кусочной функции; bisect(a, b, eps) – метод бисекции для поиска корня; findRoots(start, end, step) – поиск всех корней на интервале перебором знаков; checkAnswer(task, variant, answer) – сравнение ответа пользователя с эталоном.

Класс DataBase (Singleton)

Реализован паттерн улучшённого Singleton с вспомогательным классом DataBaseDestroyer для корректного удаления при завершении программы. Пароли хранятся только в виде SHA-256 хэшей. При регистрации записывается 6-значный код подтверждения. Данные хранятся в /app/data/server.db внутри Docker volume.

Класс EmailSender

Реализует отправку электронной почты через прямое SMTP-соединение по TCP без сторонних библиотек. Устанавливается соединение с SMTP-сервером (MailHog на порту 1025), выполняется рукопожатие EHLO, команды MAIL FROM / RCPT TO / DATA, затем тело письма и завершающая точка.

Клиентская часть (Client)

Клиент реализован как Qt GUI-приложение с несколькими оконными экранами, управляемыми через QStackedWidget. Навигация между экранами осуществляется через MainWindow:

Класс	Назначение
TcpClient (Singleton)	Единственный сетевой объект. Управляет TCP-соединением, отправляет сообщения и эмитирует сигнал responseReceived при получении ответа.
MainWindow	Центральный контроллер. Содержит QStackedWidget, обрабатывает все сигналы от дочерних окон, формирует TCP-сообщения и разбирает ответы сервера.
LoginWindow	Экран авторизации: поля логина и пароля, кнопки входа и перехода к регистрации.
RegisterWindow	Двухэтапный экран: ввод данных → ввод кода подтверждения (QStackedWidget внутри).
TaskWindow	Основной рабочий экран: выбор задачи, ввод ответа, просмотр статистики, переход к графикам.
GraphWindow	Вкладки с тремя графиками кусочных функций, построенными с помощью библиотеки QCustomPlot.

3.4. Протокол обмена

Все сообщения передаются в текстовом формате, разделены символом \n. Параметры внутри сообщения разделены символом &:

Запрос клиента	Ответ сервера	Описание
auth&login&password	auth+&login / auth-	Авторизация
reg&login&password&email	reg+&login / reg-	Регистрация
confirm&login&code	confirm+ / confirm-	Подтверждение email
stat&login	stat&N\$M&P	Статистика (верные\$попытки&очки)
check&task&variant&answer	check+ / check-	Проверка ответа
getparams	params&a1&b1&...	Получить параметры функции
setparams&a1&b1&...	params_set+ / params_set-	Изменить параметры функции

4. Применённые технологии

Qt Framework 6

Вся разработка ведётся на C++ с использованием фреймворка Qt версии 6.10. Qt предоставляет кросс-платформенную абстракцию для сетевого взаимодействия, работы с базами данных, построения GUI и тестирования – что позволяет использовать единый инструментарий для всех компонентов системы.

Модуль Qt	Применение	Ключевые классы
Qt Network	TCP-сервер и клиент, SMTP-соединение	QTcpServer, QTcpSocket
Qt SQL	Работа с SQLite базой данных	QSqlDatabase, QSqlQuery, QSqlError
Qt Core	Строки, коллекции, файлы, генератор случайных чисел	QString, QMap, QVector, QFile, QRandomGenerator, QCryptographicHash
Qt Widgets	Графический интерфейс клиента	QStackedWidget, QLineEdit, QPushButton, QComboBox, QLabel
Qt Test	Модульное тестирование	QTest, QVERIFY, QCOMPARE, QFETCH

4.2. Паттерны проектирования

Singleton

Применён для двух ключевых объектов системы. Реализован через приватный конструктор, статический метод getInstance() и вспомогательный класс-уничтожитель для корректного освобождения памяти при завершении программы.

– DataBase – единственный экземпляр соединения с SQLite. Исключает дублирование открытых соединений и гарантирует единственность состояния.

– TcpClient – единственный сетевой объект клиента. Все окна получают один и тот же экземпляр, что предотвращает создание параллельных соединений.

MVC (Model-View-Controller)

Архитектурный паттерн применён как на сервере, так и на клиенте, разделяя ответственности между слоями:

– Сервер: MyTcpServer (View) → functionsforserver (Controller) → DataBase (Model)

– Клиент: LoginWindow / TaskWindow / ... (View) → MainWindow (Controller) → TcpClient (Model)

Signal–Slot (Наблюдатель)

Механизм сигналов и слотов Qt – реализация паттерна Observer. Используется для: уведомления MainWindow об ответах сервера через сигнал responseReceived; передачи пользовательских событий (loginRequested, checkRequested) от View к Controller; обработки сетевых событий (connected, disconnected, errorOccurred).

4.3. SQLite

Встроенная реляционная база данных, не требующая отдельного сервера. Файл базы данных хранится в Docker volume /app/data/server.db, что обеспечивает персистентность данных между перезапусками контейнера.

Модуль Qt	Применение	Ключевые классы
Qt Network	TCP-сервер и клиент, SMTP-соединение	QTcpServer, QTcpSocket
Qt SQL	Работа с SQLite базой данных	QSqlDatabase, QSqlQuery, QSqlError
Qt Core	Строки, коллекции, файлы, генератор случайных чисел	QString, QMap, QVector, QFile, QRandomGenerator, QCryptographicHash
Qt Widgets	Графический интерфейс клиента	QStackedWidget, QLineEdit, QPushButton, QComboBox, QLabel

Qt Test	Модульное тестирование	QTest, QVERIFY, QCOMPARE, QFETCH
---------	------------------------	----------------------------------

4.4. SMTP и MailHog

Подтверждение регистрации реализовано через отправку письма с 6-значным кодом. SMTP-соединение устанавливается напрямую через QTcpSocket без сторонних библиотек. Для разработки и тестирования используется MailHog – инструмент-перехватчик SMTP, запускаемый в Docker:

Порт 1025 – принимает SMTP-соединения от сервера.

Порт 8025 – веб-интерфейс для просмотра перехваченных писем (<http://localhost:8025>)

4.5. Docker и Docker Compose

Использование Docker решает следующие задачи:

- Изоляция сервера от хост-системы
- Автоматический перезапуск контейнеров после перезагрузки компьютера (restart: unless-stopped)
- Персистентность данных через именованный volume (db_data:/app/data)
- DNS-маршрутизация между контейнерами по именам сервисов (mailhog:1025)

4.6. Qt Test – модульное тестирование

Набор unit-тестов реализован в отдельном субпроекте Tests с использованием фреймворка QTest. Тесты охватывают все ключевые функции серверной логики:

Группа тестов	Функция	Что проверяется
Парсинг протокола	parsing()	Невалидные команды → "error"; команды с неполными параметрами → соответствующий отказ
Вычисление функции	computeF()	Значения на всех трёх сегментах, граничные точки, NaN для $x \leq -1$
Метод бисекции	bisect()	Корректность найденных корней с точностью 0.001 для всех трёх корней
Проверка ответов	checkAnswer()	Правильные и неправильные ответы для задач 1, 2, 3; граничная точность ± 0.001

4.7. Doxygen

Документирование кода выполнено в формате Doxygen-комментариев (\brief, \param, \return, \details). Конфигурационный файл Doxyfile настроен для генерации единой HTML-документации по всем трём компонентам (EchoServer, Client, Tests). Документация включает описания всех классов, методов, параметров, а также исходный код с синтаксической подсветкой.

5. Тестирование

5.1. Стратегия тестирования

Проект охватывает три уровня тестирования:

Уровень	Инструмент	Охват
Модульный (Unit)	Qt Test (QtTest)	parsing(), computeF(), bisect(), checkAnswer()
Интеграционный	Putty (Raw TCP)	Протокол обмена, регистрация, авторизация, подтверждение email
Системный	Ручной (Client ↔ Server)	Полный пользовательский сценарий от регистрации до просмотра статистики

5.2. Ключевые тест-кейсы

ID	Описание	Входные данные	Ожидаемый результат	Тип
TC-001	Неизвестная команда	"ewqeewqewq"	"error"	Unit
TC-002	auth с одним параметром	"auth&login"	"auth-"	Unit
TC-003	computeF на сегменте 1	x = -0.5	$\ln(0.5) \approx -0.693$	Unit
TC-004	computeF корень сег. 3	x = 3.0	0.0	Unit
TC-005	bisect первый корень	[0.0, 0.7]	$\approx 0.347 \pm 0.001$	Unit
TC-006	checkAnswer верный	task=1, ans="0.347"	true	Unit
TC-007	checkAnswer неверный	task=1, ans="0.5"	false	Unit
TC-008	Регистрация через Putty	reg&test&pass&m@m.com	reg+&test	Интегр.
TC-009	Авторизация без confirm	auth&test&pass	auth-	Интегр.
TC-010	Проверка ответа задача 3	check&3&1&3.0	check+	Интегр.

6 Заключение

В ходе выполнения проекта разработана полноценная клиент-серверная система для решения математических задач. Проект охватывает широкий спектр технологий и подходов, применяемых в реальной разработке программного обеспечения.

- Реализованные компоненты:
- Консольный TCP-сервер на Qt с архитектурой MVC
- Графический Qt-клиент с навигацией между экранами и визуализацией графиков
- Система регистрации с подтверждением по email через SMTP
- Хранение данных в SQLite с хэшированием паролей
- Численный метод нахождения корней (бисекция) с точностью 10^{-6}
- Модульные тесты с покрытием ключевых функций
- Docker-контейнеризация с MailHog для тестового SMTP
- Автоматическая документация через Doxygen

Приобретённые навыки в ходе выполнения проекта:

- Проектирование клиент-серверных приложений на C++/Qt
- Применение архитектурных паттернов (MVC, Singleton, Observer)
- Работа с сетевым стеком Qt (QTcpServer, QTcpSocket)
- Интеграция с реляционными базами данных (Qt SQL, SQLite)
- Реализация SMTP-протокола на уровне TCP
- Контейнеризация приложений с Docker и Docker Compose
- Написание unit-тестов с Qt Test
- Документирование кода в стандарте Doxygen